



Instituto Politécnico Nacional



Unidad Profesional Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas

Unidad de Aprendizaje: Inteligencia Artificial

Particle Swarm Optimization

Realizado por:

Yañez Gonzalez Diego

2021640575

Delgado Hernández Emiliano

2021301920

Reyes Leal Leonardo

2022640240

Docente:

Anzueto Ríos Alvaro

Grupo

3BM15

25 de noviembre de 2024

OBJETIVO

Desarrollar e implementar una red neuronal destinada al seguimiento de señales, optimizando su matriz de pesos y polarizaciones (biases) mediante el algoritmo de Particle Swarm Optimization (PSO). Esta tarea busca fortalecer la comprensión de la optimización de redes neuronales y la aplicación práctica de algoritmos metaheurísticos como PSO.

INTRODUCCIÓN

En un principio los diferentes métodos de optimización fueron concebidos para elaborar modelos de conductas sociales, como el movimiento descrito como los organismos vivos en una bandada de aves o un banco de peces.

En la mayoría de las aplicaciones de la ingeniería, ya sea en los sistemas de procesamiento de señales o de energía eléctrica, requieren algoritmos eficaces y eficientes para que puedan resolver sistemas de optimización relacionados en su área. Los problemas de optimización a lo largo de los diferentes problemas que suceden en la vida real se han resuelto por ejemplo con Particle Swarm Optimization (PSO) y la optimización de colonias (ACO), así como diferentes algoritmos metaheurísticos.

MARCO TEÓRICO

El algoritmo PSO trabaja con una población (llamada nube o enjambre) de soluciones llamadas partículas. Dichas partículas se desplazan a lo largo del espacio de búsqueda conforme a ciertas reglas matemáticas. El movimiento de cada partícula depende de su mejor posición obtenida, así como la mejor posición global hallada en todo el espacio de búsqueda. A medida que se descubren nuevas y mejores posiciones, éstas pasan a orientar movimientos de las partículas. Además de que todas las partículas su mejor rendimiento individual y también conocen el mejor rendimiento global. Ajustan su velocidad teniendo en cuenta su mejor rendimiento y también teniendo en cuenta el mejor rendimiento de la mejor partícula. El proceso se repite con el objetivo, que no es siempre garantizado, de hallar en algún momento una solución que puede llegar a ser suficientemente satisfactoria.

Función de velocidad: $V_{t+1} = WV_t + C_1 * r_1(P_t - X_t) + C_2 * r_2(G_t - X_t)$

Función de posición: $X_{t+1} = X_1(t) + V_{t+1}$

Donde: W = *Coeficiente de Inercia*
 C_1 y C_2 = *Coeficientes de Aceleración*
 P_t = *Mejor posición conocida de la Partícula*
 G_t = *Mejor posición global conocida.*
 r_1 y r_2 = *valor aleatorio para movimiento de las partículas.*

Su funcionamiento se basa en que cada partícula tiene su propia posición, que en el caso de dos dimensiones vendrá determinado por un vector x,y , en el espacio de búsqueda y además tendrá una velocidad que de igual forma vendrá dado de la forma (x,y) .

Las partículas en la vida real, tienen una cantidad de inercia, en este caso sería W , lo que las mantiene en la misma dirección en la que se movían así como una aceleración que sería el cambio de la velocidad y que esta depende de dos características:

1. Cada partícula es atraída hacia la mejor posición para ella, que personalmente se convertiría en una P_t^{best} .
2. Cada partícula es atraída hacia la mejor posición que ha sido encontrada por el conjunto de partículas en el espacio de búsqueda P_G^{best} .

La fuerza con la que las partículas son empujadas en las diferentes direcciones dependen de los parámetros C_1 y C_2 de tal manera de que las partículas se alejan de estas mejores localizaciones, la fuerza de atracción es mayor y también se suele incluir un factor aleatorio que influye en cómo es que las partículas son empujadas hacia estas dos localizaciones ($r1$ y $r2$).

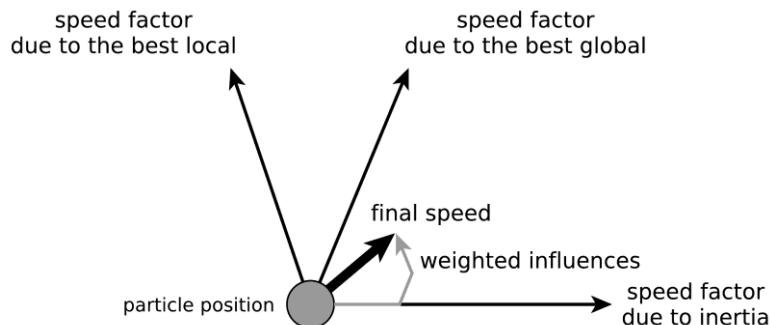


Figura 1. Influencia de las velocidades sobre la posición de la partícula.

DESARROLLO

Pseudocódigo del programa:

Inicializar parámetros:

- Número de variables, coeficientes de aceleración ($c1$, $c2$), coeficiente de inercia (w)
- Número máximo de iteraciones (max_iter), tamaño de la población ($poblacion$).
- Límites de las variables (var_min , var_max)
- Patrón de entrada ($pattern$) y factor de disminución de w

(disminu)

- Lista para registrar el error en cada iteración

Definir funciones auxiliares:

- 'purelin(n)': Calcula una salida lineal basada en la entrada 'n'.
- 'logsig(n)': Calcula la salida de una función sigmoide logística basada en 'n'.
- 'neurona(x, pattern)': Simula una red neuronal con dos neuronas de entrada y una de salida. Calcula el error promedio cuadrático.
- 'fitness(pattern, particula)': Calcula el error cuadrático medio (MSE) entre la salida de la red neuronal y la salida esperada.

Inicializar estructura de partículas:

- Cada partícula tiene posición, velocidad, costo, mejor posición local y mejor costo local.
- Inicializar las posiciones aleatoriamente entre 'var_min' y 'var_max'.
- Inicializar velocidades a cero.
- Evaluar el costo inicial de cada partícula usando la función de fitness.
- Actualizar las mejores posiciones locales y el mejor global basado en los costos.

Para cada iteración hasta 'max_iter':

- Para cada partícula:
 - Actualizar la velocidad usando la ecuación de PSO:
velocidad = w * velocidad
+ c1 * rand * (mejor_posicion_local - posicion_actual)
+ c2 * rand * (mejor_posicion_global - posicion_actual)
 - Actualizar la posición de la partícula:
posicion = posicion + velocidad
Restringir la posición entre 'var_min' y 'var_max'.
 - Calcular el nuevo costo de la partícula.
 - Si el nuevo costo es mejor que el mejor costo local:
 - Actualizar la mejor posición local y el mejor costo local.
 - Si el mejor costo local es mejor que el mejor costo global:
 - Actualizar la mejor posición global y el mejor costo global.
 - Reducir 'w' multiplicándolo por 'disminu'.

- Registrar el mejor costo global en cada iteración.

Visualizar los resultados:

- Graficar el costo global en cada iteración.
- Simular la red neuronal usando la mejor posición global encontrada.
- Graficar la salida de la red neuronal frente a la salida esperada y el error.

Mostrar gráficas finales.

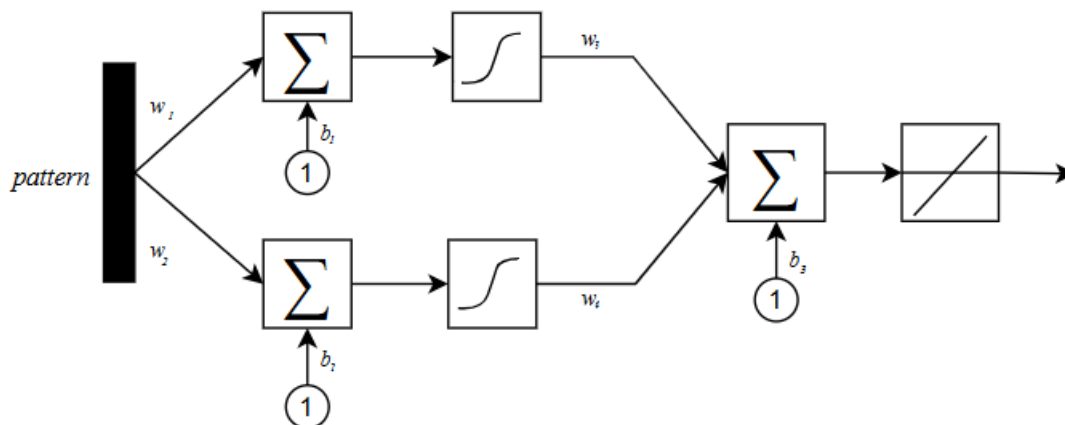


Figura 2. Diseño de la red neuronal propuesta para la solución del problema.

RESULTADOS

La función objetivo a seguir fue

$$y = 1 + \sin\left(\frac{\pi}{4}x\right)$$

Los parámetros utilizados se muestran a continuación:

coeficientes de aceleración $c_1 = c_2 = 1.5$

coeficiente de inercia $w = 1$

número máximo de iteraciones : 100

población : 30

dominio de la función: $[-2, 2]$ con paso de 0.2

factor de disminución : 0.5

Ejecutando el programa con los parámetros anteriores, los resultados obtenidos se muestran en las gráficas de las figuras 3 y 4.

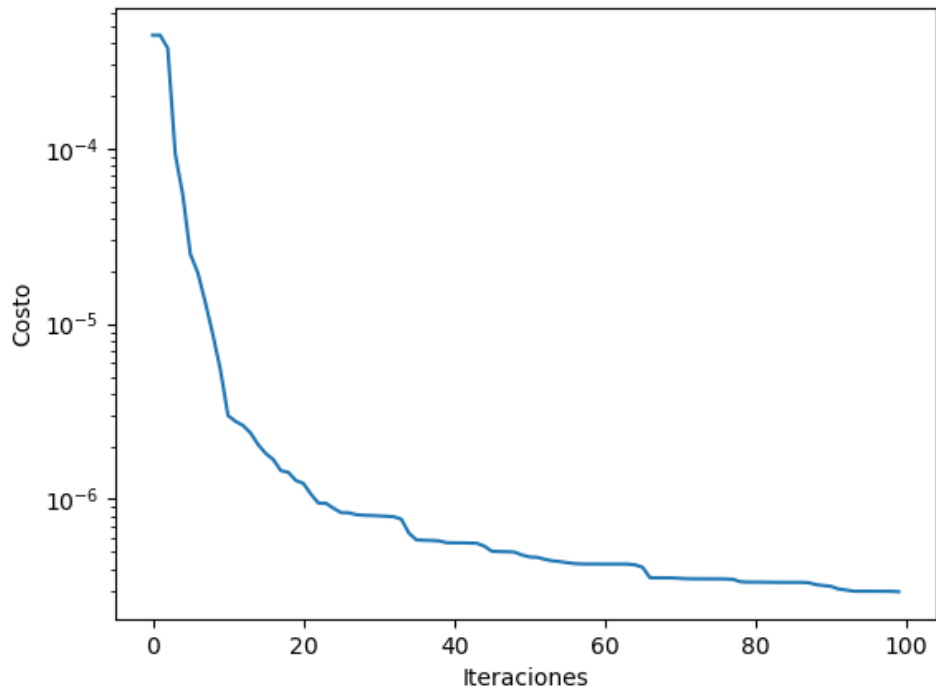


Figura 3. Evolución del costo contra el número de iteraciones.

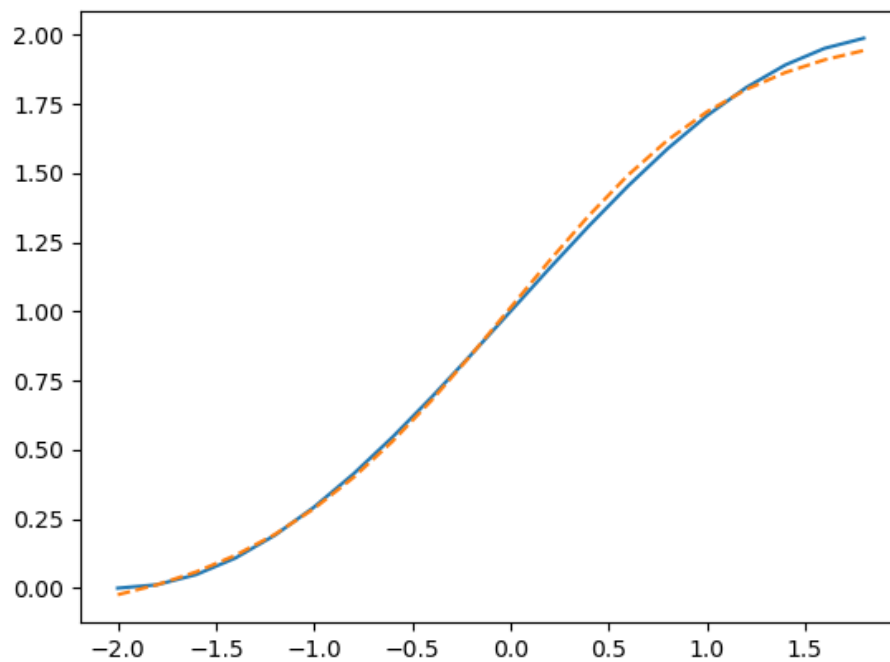


Figura 4. Evaluación del vector de entrada con los mejores valores obtenidos.

Los mejores valores para las variables de nuestro problema que arrojaron los

resultados de las gráficas en las figuras 3 y 4, son :

$$\begin{bmatrix} w_1 \\ w_2 \\ b_1 \\ b_2 \\ w_3 \\ w_4 \\ b_3 \end{bmatrix} = \begin{bmatrix} -1.94078555 \\ -1.51938788 \\ 0.45538564 \\ -0.54840155 \\ -0.9179854 \\ -1.24299509 \\ 2.03030476 \end{bmatrix}$$

$$\text{costo} = 2.9595683323893403 \times 10^{-7}$$

CONCLUSIONES

Emiliano Delgado Hernández:

A lo largo de ésta práctica se pudo demostrar de forma práctica y eficaz el algoritmo de optimización por enjambre de partículas ("Particle Swarm Optmiization"[PSO]), el cuál puede incluso (para ésta demostración en específico) simular una red neuronal multicapa con 2 neuronas en la capa oculta y una neurona de salida, para la realización de un seguidor de señal, de forma certera, evitando así el entrenamiento por épocas y por alguna otra ecuación de aprendizaje, dicho algoritmo se dedica a solucionar problemas con una ecuación lineal, de forma "romántica" utilizando una sugestión personal y una social, dónde se puede tomar distintos porcentajes de ésta sugestión y vectorialmente hablando modificar la posición de la partícula según la velocidad que se desarrolle conforme cada coeficiente. Se logra observar con éste caso particular como el PSO es capaz de replicar un entrenamiento de varias épocas, en menor número de iteraciones, consiguiendo una solución con posiblemente una convergencia con mayor rapidez que una red neuronal tan pequeña como lo es la planteada, Sin embargo, sí que se notó a lo largo del desarrollo de la misma que es dependiente de las mismas condiciones que la red, por usar las funciones de activación de la misma, pues al tener pesos inicializados con una magnitud relativamente grande (mucho mayor a 1) puede llegar a indeterminación, o por otro lado conseguir una saturación con alta rapidez según sea la función utilizada (para éste caso se utilizó purelin y logsig y se consiguieron resultados decentes). Por otro lado, también se notó que con rangos pequeños de la señal se obtienen resultados impresionantes, no obstante, al momento de incluir mayor número de datos en la parte del target, se llega a complicar la tarea para el PSO.

Leonardo Reyes Leal:

Al implementar el algoritmo PSO para encontrar los valores que permiten a una red neuronal, seguir una señal, podemos comprobar la versatilidad de éste. Observamos que para un dominio pequeño de la función, el programa encontró una solución bastante aproximada en pocas iteraciones, sin embargo al tratar de ampliar el dominio, fue más complicado para la red el seguir la señal.

Este comportamiento también se presentó al aumentar el número de elementos en el patrón de entrada, es decir, cuando el paso entre elementos del dominio de la función era menor. Para pocos datos de entrada y un dominio en el que la función presentaba pocos cambios, la red neuronal propuesta pudo seguir adecuadamente la señal.

Por medio de este tipo de algoritmos, podemos encontrar las soluciones óptimas de una función sin necesidad de calcular la derivada, lo que permite resolver problemas complejos de una manera sencilla inspirada en comportamientos naturales de diferentes especies. No obstante, debe tomarse en cuenta que este algoritmo utiliza ecuaciones lineales para realizar la aproximación, por lo que el tiempo de cómputo puede ser mayor. Además, la solución que encuentra puede estar más alejada de la solución real en comparación con la que encontraría un algoritmo que utilice ecuaciones no lineales para realizar la aproximación.

Diego Yañez González:

En esta práctica pudimos explorar cómo funciona el algoritmo de Particle Swarm Optimization (PSO) y aplicarlo para resolver un problema de optimización en una red neuronal sencilla. Aprendimos que las partículas, que representan posibles soluciones, se mueven en el espacio de búsqueda guiadas tanto por su experiencia personal (el mejor resultado que han encontrado) como por la experiencia del grupo (el mejor resultado global).

Fue interesante ver cómo los parámetros del algoritmo, como los coeficientes de aceleración y el de inercia, afectan el equilibrio entre buscar nuevas soluciones y aprovechar las que ya parecen prometedoras. Con el paso de las iteraciones, las partículas lograron acercarse a una solución óptima, minimizando el error y cumpliendo con el objetivo planteado.

También descubrimos que el PSO es una herramienta bastante flexible. En este caso, lo usamos para simular una red neuronal multicapa con una configuración muy sencilla (dos neuronas en la capa oculta y una en la salida) y logramos que realizara tareas como el seguimiento de señales, pero sin el entrenamiento tradicional basado en épocas o reglas de aprendizaje complejas. En lugar de eso, el algoritmo resolvió el problema de manera eficiente utilizando su enfoque colaborativo: cada partícula tomaba en cuenta su propia experiencia y la del grupo, ajustando su posición en función de estos factores.

Sin embargo, notamos algunas limitaciones. Por ejemplo, si los pesos iniciales de la red tienen valores muy grandes o si no elegimos bien las funciones de activación, el algoritmo puede presentar problemas, como saturarse muy rápido o incluso fallar en converger. Además, aunque PSO funciona muy bien con señales pequeñas, puede tener dificultades cuando el tamaño del conjunto de datos objetivo aumenta demasiado.

En resumen, esta práctica nos permitió ver el potencial del PSO como una herramienta poderosa, capaz de resolver problemas complejos de manera colaborativa e inspirada en el comportamiento de los sistemas naturales. Nos dejó claro que su versatilidad lo hace aplicable en muchas áreas, desde el aprendizaje automático hasta problemas prácticos de ingeniería.

REFERENCIAS

- Particle Swarm Optimization: A Comprehensive Survey. (2022). IEEE Journals & Magazine | IEEE Xplore. <https://ieeexplore.ieee.org/document/9680690>
- Rishi. (2024, 18 noviembre). Understanding Particle Swarm Optimization (PSO): From Basics to Brilliance. Medium.
<https://thisisrishi.medium.com/understanding-particle-swarm-optimization-pso-from-basics-to-brilliance-d0373ad059b6>